CSL301 : Operating System
LAB- 5
100 Marks

Problem -1 **25 Marks**

Design and implement an **Interactive Event/Notification System** using **System V Shared Memory**.

The system consists of two independent processes:

- **Event Generator** — accepts events from the user and places them in shared memory.
- **Event Monitor** — continuously checks shared memory and displays new events as soon as they arrive.

The user should be able to enter events such as:

CPU_HIGH
MEMORY_HIGH
LOGIN
FILE_CREATED
PROCESS_STARTED
PROCESS_TERMINATED

Each event should contain:

- Event ID
- Event type
- Event message

For example:

Enter Event Type: CPU_HIGH
Enter Event Message: CPU usage exceeded 90%

Requirements

1. Use **System V shared memory**.

2. Use ftok(), shmget(), shmat(), shmdt(), and shmctl().
3. The Event Generator must accept events from the keyboard.
4. The Event Monitor must display events received through shared memory.
5. The Generator should not overwrite an event until the Monitor has processed it.
6. The Monitor should wait when there is no new event.
7. Provide an option to terminate the Event Generator.
8. The Monitor should terminate only after all pending events have been processed.
9. Properly remove the shared-memory segment when the system terminates.
10. Use two terminal windows, one for event_monitor and another for event_generator.

Problem-2 **25 Marks**

Interactive Sleeping Barber using Shared Memory

A barber shop has one barber and a limited number of waiting chairs.

The following rules apply:

1. If there are no customers, the barber sleeps.
2. When a customer arrives and the barber is free, the barber serves the customer.
3. If the barber is busy, the customer waits in a chair.
4. If all waiting chairs are occupied, the arriving customer leaves.
5. The barber serves waiting customers one by one.
6. The customer process should accept customer IDs from the user.
7. The customer should enter -1 to stop generating customers.
8. Shared memory must be used to maintain the waiting queue and barber-shop state.
9. Use two terminal windows in this program also.

Problem-3 Parent–Child Message Service **25 Marks**

- Write a C program using an **anonymous pipe** and fork() where the parent and child communicate with each other.
- The **parent process** should accept a message from the user and send it to the child through a pipe.
- he **child process** should read the message, display it, and send an acknowledgement back to the parent through a second pipe.
- The parent should then display the acknowledgement.
- The user should be able to enter exit to terminate the communication.

Problem Statement-4 **25 Marks**

Create a **parent-child process system using pipes**.

The **parent process** acts as a job producer. It accepts jobs from the user. Each job contains:

- **Job ID**
- **Operation**
- **Input value**

The parent sends each job to the child through a **pipe**.

The **child process** acts as a job processor. It receives the job, performs the requested operation, and sends the result back to the parent through a **second pipe**.

The program should continue accepting jobs until the user enters **-1 as the Job ID**.

Supported operations

Operation	**Meaning**	**Example**
square	Calculate square	5 → 25

cube	Calculate cube	3 → 27
double	Calculate 2 × value	8 → 16
factorial	Calculate factorial	5 → 120

Submission:

- Codes files.
- Output screenshots.
- All in a single zip file.